

Contents

Specification Review Report — chapter4 eval harness (labs/week1/chapter4/SPEC.md)	1
1. Executive Summary	2
2. Overall Maturity	2
3. Findings Summary	2
4. Detailed Findings	4
F-001 — Ambiguity in verdict/judge ownership	4
F-002 — Verdict-status enum conflict between ch3 and ch4	4
F-003 — <code>dataset_id</code> resolved inconsistently with the §28 append-then-compare loop	5
F-004 — Dataset-size floor (50–100+) contradicts the test suite’s 5-row fixtures	5
F-005 — Gates constraint units and parameter pairing undefined	5
F-006 — <code>run</code> behavior on invalid dataset unspecified	6
F-007 — <code>PARSE_BLOCKED</code> metric semantics undefined	6
F-008 — Classification step 2 (label-evidence <code>EVALUATION_FAILURE</code>) temporally ambiguous	7
F-009 — ch3 MRR/MAP/NDCG lineage vs <code>METRIC_KEYS</code> mismatch	7
F-010 — Global <code>--model</code> flag ambiguity (generation vs judge)	7
F-011 — <code>pair</code> config JSON schema unspecified	8
F-012 — Test-id collisions and traceability gaps	8
F-013 — <code>usage_kind</code> mismatch unguarded at compare-time	9
F-014 — <code>run</code> exit-code row omits the K-01 usage-error code	9
F-015 — <code>by_difficulty</code> asymmetric presence unspecified	9
5. Requirements Review	10
6. Interface and Data-Contract Review	10
7. State and Failure Review	10
8. Determinism and Algorithm Review	11
9. Edge-Case Review	11
10. Non-Functional Requirement Review	11
11. Security and Trust-Boundary Review	11
12. Observability and Provenance Review	11
13. Testing and Verification Review	12
14. Metrics and Evaluation Review	12
15. Traceability Review	12
16. Internal-Consistency Review	12
17. Architecture Review	12
18. Implementation-Agent Readiness	13
19. Quality Scorecard	13
20. Remediation Plan	14
P0 — Blocking (must be resolved before implementation)	14
P1 — Important	14
P2 — Improvement	14
21. Final Verdict	15

Specification Review Report — chapter4 eval harness (labs/week1/chapter4/SPEC.md)

- **Subject of review:** labs/week1/chapter4/SPEC.md (v0.1, eval harness for the ch3 RAG pipeline).
- **Method:** four-pass review per the `spec-review` skill — Pass 1 comprehension, Pass 2 local precision, Pass 3 cross-consistency, Pass 4 implementation simulation — across the skill’s twenty dimensions; findings labeled `CRITICAL/HIGH/MEDIUM/LOW`; scorecard 0–5; maturity Level 0–4.
- **Summary:** Level 2 (Implementable). 15 findings: **0 CRITICAL, 4 HIGH, 7 MEDIUM, 4 LOW**. Readiness: **NOT READY** — two semantic ambiguities (judge ownership, verdict-status mapping) must be resolved before implementation.

1. Executive Summary

The SPEC is a strong v0.1: the scope boundary (an eval harness that *wraps* the ch3 RAG pipeline through a pinned adapter rather than re-implementing it) is crisp, the requirement set is observable-behavior oriented, the artifact-level architecture (`dataset_report.json`, `eval.json`, `compare_report.json`, `gate_report.json`, `judge_check_report.json`, `pair_report.json`) makes provenance first-class, and the deterministic-before-probabilistic ordering (`checks` $\rightarrow$ `judge` $\rightarrow$ `classifier`) is the correct encoding of ch4 §5–§7. Stratification (`by_category`), direction-aware Δ tables, hard-constraint gates, and label-anchored evaluator validity are all normative and well-formed.

The review finds **15 findings** — **4 HIGH, 7 MEDIUM, 4 LOW** (no CRITICAL). The four HIGH findings cluster on the seam between the AoE (ch3) and the ch4 eval core: (F-001) who *owns* the verdict — the AoE’s internal judge or the harness’s own; (F-002) the verdict-status enum mapping between ch3’s **PARTIAL**-capable status and ch4’s `{PASS, FAIL, PARSE_BLOCKED}`; (F-003) `dataset_id`’s “stable name/hash” ambiguity versus the §28 production loop that must be able to append cases and still compare; (F-004) the unfloorable 50–100+ dataset-size rule versus the 5-row fixtures the test suite uses. Until F-001/F-002 are resolved, two implementers can write substantively different verdict pipelines — hence **NOT READY** (Level 2 — Implementable).

Strengths most worth preserving: explicit index/query flag partition (I-008/R-08), zero-denominator semantics per metric (I-001/E-03), byte-identity formatting rules (I-002), fail-closed gates with explicit `n/m` markers (E-07/E-11), and the label-evidence-only rule for **EVALUATION_FAILURE** (I-017/E-16).

Weakest seams: verdict pipeline ownership and status enum (F-001/F-002), gates unit semantics (F-005), `PARSE_BLOCKED` metric treatment (F-007), classification timing versus label ingestion (F-008), and a handful of naming/traceability hygiene items (F-012).

2. Overall Maturity

Level 2 — Implementable. A competent engineer can build the harness end-to-end on the mock path, and conformance of the deterministic core is largely objectively testable today (I-002/I-003/I-004). The blockers preventing Level 3 are the unresolved high-severity seam issues above; the document is explicit about being a pre-review v0.1 awaiting this pass, which is consistent with the ch3 workflow (v0.1 $\rightarrow$ review $\rightarrow$ v0.2). Level 3 requires the P0 items from the remediation plan applied.

3. Findings Summary

Severity	Count	IDs
CRITICAL	0	—
HIGH	4	F-001, F-002, F-003, F-004
MEDIUM	7	F-005, F-006, F-007, F-008, F-009, F-010, F-011
LOW	4	F-012, F-013, F-014, F-015

ID	Severity	Subject	Location
F-001	HIGH	Verdict/judge ownership ambiguity between AoE result and harness evaluator	C-02 vs C-03
F-002	HIGH	Verdict-status enum mapping conflict (ch3 PARTIAL vs ch4 <code>{PASS, FAIL, PARSE_BLOCKED}</code>)	C-03, I-005

ID	Severity	Subject	Location
F-003	HIGH	<code>dataset_id</code> “stable name/hash” undefined → blocks §28 append-then-compare workflow	C-01, E-12
F-004	HIGH	50–100+ dataset-size rule cannot bind <code>check</code> (tests use 5-row fixtures)	R-02, T-01
F-005	MEDIUM	Gates unit semantics (<code>max_pct_points</code> vs <code>max_pct</code>) and constraint-parameter pairing undefined	C-07
F-006	MEDIUM	<code>run</code> behavior on an invalid dataset unspecified (E-02 scoped to <code>check</code> only)	§5.1, E-02 §9.1
F-007	MEDIUM	Metric semantics for <code>PARSE_BLOCKED</code> verdicts (accuracy/completeness) undefined	C-04, I-005
F-008	MEDIUM	Classification step 2 (label-evidence <code>EVALUATION_FAILURE</code>) ordering vs ingestion undefined	C-08, §3.2
F-009	MEDIUM	MRR/MAP/NDCG reused from ch3 but absent from <code>METRIC_KEYS</code> — report coverage ambiguous	§0, §1, C-04
F-010	MEDIUM	Global <code>--model</code> flag ambiguous (generation model vs judge model vs both)	§5.1, R-15
F-011	MEDIUM	<code>pair</code> config JSON schema unspecified (gates.yml has schema; pair configs do not)	C-10, §5.1
F-012	LOW	Test-id collisions (T-08a, T-13) and untested edge cases (E-01, E-05, E-08, E-14, E-17)	§9, §11
F-013	LOW	<code>usage_kind</code> synthetic-vs-measured mismatch unguarded at <code>compare</code>	C-02, C-06
F-014	LOW	<code>run</code> exit-code row omits the K-01 usage-error code	§5.1 vs K-01
F-015	LOW	<code>by_difficulty</code> asymmetric presence in <code>compare</code> unspecified	R-05, C-06

4. Detailed Findings

F-001 — Ambiguity in verdict/judge ownership

Severity: HIGH

Location: §4 C-02 (AoEResult carries `verdict`) vs §4 C-03 (evaluator defines `MockJudge/OllamaJudge`)

Observation

C-02's `AoEResult` includes a `verdict: dict` produced inside the AoE (the ch3 pipeline already runs its own `judgment.py` internally). C-03, however, declares the harness evaluator itself to be “DeterministicChecks first, then `MockJudge / OllamaJudge`”. Whether ch4's `evaluator.py` uses the AoE-provided verdict as-is, overrides it with its own judges, or deletes/duplicates the ch3 judges is unspecified.

Why it matters

Two implementers can build materially different verdict pipelines: one treats the AoE verdict as authoritative and adds only deterministic checks; the other re-judges post-hoc, producing different metrics under label-validation.

Potential consequence

T-08a (metric lineage), T-17 (judge-check), and metrics correctness diverge between implementations; evaluator validation (§26) targets the wrong component.

Recommended resolution

Pin the contract: the `AoEResult.verdict` is the evaluated verdict (per the ch3 pinned interface); `evaluator.py` adds deterministic checks and the failure-stage mapping only; the `MockJudge/OllamaJudge` classes in `evaluator.py` are aliases/wrappers around ch3's `judgment.py` for judge-check validation purposes, not a second verdict path.

F-002 — Verdict-status enum conflict between ch3 and ch4

Severity: HIGH

Location: §4 C-03 (“status set: `PASS | FAIL | PARSE_BLOCKED` (I-005)”) vs ch3 verdict's richer status semantics (ch3 E-11 `PARTIAL`)

Observation

The ch3 verdict record carries a `status` field with documented `PARTIAL` semantics (ch3 spec, E-11/E-12). Ch4 restricts its own status set to `{PASS, FAIL, PARSE_BLOCKED}` and later consumes `correct/-`-based metrics under that assumption. The mapping from ch3's enumerable statuses to ch4's is undefined (is `PARTIAL` mapped to `FAIL`? dropped? preserved?).

Why it matters

Metrics computations (`accuracy = mean(verdict.correct)`) and failure classification (C-08) depend on known enum mapping. A `PARTIAL` verdict is treated inconsistently by different implementers.

Potential consequence

Aggregate accuracy numbers differ; classification precedence step 1 vs step 5 disagrees.

Recommended resolution

Define a total enum-mapping table in C-03 (e.g. `PASS → PASS, FAIL → FAIL, PARTIAL → FAIL-with-preserved-nuance-flag`, `PARSE_BLOCKED` introduced by ch4 only) and assert on load (I-010).

F-003 — `dataset_id` resolved inconsistently with the §28 append-then-compare loop

Severity: HIGH

Location: §4 C-01 (“stable name/hash”) + §8 E-12 (compare exits 2 on id mismatch)

Observation

`dataset_id` is declared as “stable name/hash” without choosing. If hash-of-content, then the §28/ §33 workflows that append a production case to the golden dataset change the hash, and `compare` between the pre- and post-append datasets deterministically exits 2 — defeating the production loop the spec is designed to support.

Why it matters

The §28 production-loop closure (R-13) is a stated requirement. E-12, as written, may block it.

Potential consequence

Users locked into either immutable datasets (no production growth) or forced `--force`-style bypasses, eroding I-012’s stratification guarantees across versions.

Recommended resolution

Resolve `dataset_id` as a **declared stable name** (hash of *dataset name*, not content); compare guards on equality of the *name*; document that case-appends are permitted (E-12 guards content *purpose*, not identity).

F-004 — Dataset-size floor (50–100+) contradicts the test suite’s 5-row fixtures

Severity: HIGH

Location: R-02 (“shall hold 50–100+”) vs §9.1 (T-01 ... “a valid 5-row dataset”)

Observation

`check` either enforces the R-02 50–100+ floor (then every fixture must be 50+ rows and T-01/T-03 fixtures are invalid) or it does not (then R-02 is unverifiable as a `check` violation). The spec never says which, nor states a “fixture-only” exemption.

Why it matters

Determinism of `check`’s exit code on small datasets differs between implementers.

Potential consequence

T-01 (valid 5-row dataset exits 0) conflicts with a strict R-02 floor enforcer (exits 3).

Recommended resolution

Phrase R-02 as a *recommendation* (“SHOULD”) and make `check` floor-independent; OR gate the floor behind a documented `--strict` flag; fix T-01 to state which mode it exercises.

F-005 — Gates constraint units and parameter pairing undefined

Severity: MEDIUM

Location: §4 C-07 (gates.yml uses `max_pct_points` and `max_pct` interchangeably) + §25 MAY (`min_value/max_value`)

Observation

C-07’s example mixes *percentage-point* bounds (`accuracy`, `drop`, `max_pct_points: 1.0`) with *relative-per-cent* bounds (`latency_p95`, `increase`, `max_pct: 20.0`). Neither unit is defined; whether `max_pct_points` pairs with `increase` and `max_pct` with `drop` (or vice versa) is unspecified; and the §25 absolute constraint MAY (`min_value/max_value`) has no validation rules (key exclusivity, direction, semantics).

Why it matters

Gate semantics (hard vs soft violations) flip numerically depending on unit interpretation.

Potential consequence

A failing gate passes in one implementation, defeating CI enforcement.

Recommended resolution

Define both units (`max_pct_points` = absolute difference vs `max_pct` = relative baseline change) with pairing validation; reject the §25 MAY keys unless one of `min_value`/`max_value` is present.

F-006 — `run` behavior on invalid dataset unspecified

Severity: MEDIUM

Location: §5.1 (check exit table) vs §8 E-02 (load errors scoped to `check`)

Observation

The only dataset-validation semantics are attached to the `check` subcommand. `run --dataset` does not say whether it revalidates first (exit 3 on violations, exit 2 on usage) or merely loads (U-undefined behavior on violations).

Why it matters

Users expect `run` to refuse bad datasets; untyped validation breaks T-01 semantics.

Potential consequence

Different implementers either revalidate (safer but slower) or never validate (silently broken `eval.jsons`).

Recommended resolution

Specify that `run` performs the same validation as `check` and returns exit 3 on violations (or explicitly defer validation to a separate phase, with a warning).

F-007 — `PARSE_BLOCKED` metric semantics undefined

Severity: MEDIUM

Location: C-04 (`accuracy = mean(verdict.correct)`) + I-005 (status totality with `PARSE_BLOCKED`)

Observation

C-04 computes per-field metrics but does not state what `correct`/`supported`/`complete` mean for a `PARSE_BLOCKED` verdict (default false? absent? zero-denominator short-circuit?).

Why it matters

A blocked parse case accidentally raises anchored metric aggregates (or wrongly suppressed).

Potential consequence

Aggregates disagree across implementations; regression gates mis-trigger.

Recommended resolution

Define that a `PARSE_BLOCKED` verdict maps `correct=false`, `supported=false`, `complete=false`, and that `unsupported_claims` counts 0 (document alongside I-001 zero-denominator rules).

F-008 — Classification step 2 (label-evidence `EVALUATION_FAILURE`) temporally ambiguous

Severity: MEDIUM

Location: §3.2 eval-time flow (no label-ingestion step) + §4 C-08 precedence step 2 + §4 C-09 label format

Observation

Classification runs during `run` (§3.2), but step 2 of the precedence says assert `EVALUATION_FAILURE` only when human-label disagreement exists. How labels reach the classifier is unspecified (labels in a second pass via `judge-check`? a backfill mechanism writing to `eval.json`? `run --labels`?).

Why it matters

Without a specified timing, the classification written into `eval.json` may be unsynchronized with the label phase; implementers choose different lifecycles.

Potential consequence

Case classifications drift between a pre-label and post-label view; T-17’s agreement check tests different things.

Recommended resolution

Specify classification-on-labels is a *post-run* phase (`judge-check --labels` recomputes and optionally emits a revised `eval.json` artifact, or remains a read-only comparison). State which.

F-009 — ch3 MRR/MAP/NDCG lineage vs `METRIC_KEYS` mismatch

Severity: MEDIUM

Location: §0 Intent (“P@k / R@k reuse of ch3 metrics.py, plus...”) + §1 Metrics actor (“add... the §19 vector”) vs §4 C-04 `METRIC_KEYS` (only accuracy, P@k, R@k, groundedness, completeness, hallucination, latency, cost)

Observation

The spec repeatedly claims metric lineage through ch3’s MRR@k / MAP / NDCG@k reuse, but the normative `METRIC_KEYS` list (that gates and compare actually operate against, per I-007) silently drops them.

Why it matters

Either the metric family is report-only (not gates-addressable) or it disappears; neither is stated.

Potential consequence

Implementers differ over whether `eval.json` aggregates must include MRR/MAP/NDCG. Gates configs with those keys fail in one interpretation (I-007 unknown key) and succeed in another.

Recommended resolution

Either extend `METRIC_KEYS` with `mrr_at_k`, `map`, `ndcg_at_k` (with direction `map`) or explicitly declare them as report-only extras and annotate §0/§1 accordingly.

F-010 — Global `--model` flag ambiguity (generation vs judge)

Severity: MEDIUM

Location: §5.1 (“Global: ... `--model` (real-path judge override)”) vs R-15 (ch3 E-13 uses `--model` for generation models) + ch3 CLI where `--model` is the generation model

Observation

Ch4’s CLI inherits ch3’s `--model` convention, then re-declares `--model` as “real-path judge override” without a counterpart generation-model flag (`--gen-model? --model-gen?`); the exact flag matrix (judge vs generation vs both) is underresolved.

Why it matters

Two implementers either apply `--model` to the judge only, generation only, or both.

Potential consequence

Real-path smoke (§9.11) targets wrong components; E-13 banner outcome changes.

Recommended resolution

Introduce explicit `--judge-model` and `--gen-model` flags (or define `--model` as generation only plus a separate `--judge-model`), and update §5.1.

F-011 — pair config JSON schema unspecified

Severity: MEDIUM

Location: §5.1 pair `--a config.json --b config.json`; C-10 has no schema reference; I-010 says schemas gate `eval/compare/gates/labels` (pair unlisted)

Observation

Pairwise evaluation configs are JSON (vs gates’ YAML) but their schematization is not declared; I-010’s own list of gated artifacts omits pair config files.

Why it matters

Untyped configs allow unknown keys to slip in silently (E-08-style rebuild confusion).

Potential consequence

Implementers disagree on which flags pair configs may contain (index-time vs query-time).

Recommended resolution

Add `schemas/pair.json` (flag key-value pairs, each either index-time or query-time) to the I-010 gated list and to §11 traceability.

F-012 — Test-id collisions and traceability gaps

Severity: LOW

Location: §9 (T-08a assigned twice — §9.3 metrics §9.9 judge validation “T-08a-reuse”; T-13 assigned twice — §9.2 GUI §9.10 pair) + §11 (E-01, E-05, E-08, E-14, E-17 not directly traced to T-ids)

Observation

Two T-ids collide and a handful of edge cases (E-01 corpus-unreadable, E-05 `run_case` exception, E-08 stale-index compare block, E-14 schema-drift rejection, E-17 GUI malformed-artifact error-path) lack dedicated test rows.

Why it matters

Test-id collisions undermine T-NN → module → behavior traceability (the §11 matrix breaks silently when names collide). Untested edge-ids weaken I/O safety guarantees.

Potential consequence

Reviewers cannot unambiguously map “which test proves E-08?”; a regression in one might pass.

Recommended resolution

Re-number (e.g. T-08a $\rightarrow$ T-08a/T-08c; T-13 $\rightarrow$ T-13/T-24), add dedicated rows for E-01, E-05, E-08, E-14, E-17, and re-export §11 after re-numbering.

F-013 — `usage_kind` mismatch unguarded at compare-time

Severity: LOW

Location: §4 C-02 (`usage_kind: synthetic | measured`) vs §4 C-06 (compare has no rule for mixed-kind artifacts)

Observation

A `synthetic`-labeled baseline compared against a `measured`-labeled current silently mixes deterministic tokens with real tokens; the Δ table may be numerically meaningless, especially on cost/ latency metrics.

Why it matters

§19's K dimension is direction-aware; comparing across kinds breaks that.

Potential consequence

Misleading regression decisions (T-09-detected regression might be an artifact).

Recommended resolution

Add a compare-time rule: warn on mixed `usage_kind` unless `--force` (or `auto-n/m` on latency/cost rows).

F-014 — `run` exit-code row omits the K-01 `usage-error` code

Severity: LOW

Location: §5.1 `run` row lists exit 0 / 4 only; K-01 declares all subcommands exit 2 on usage errors

Observation

The table formatting is inconsistent: `check`, `compare`, `gates`, `judge-check`, `pair`, `new-case` all show `... / 2 usage` whereas `run` omits 2 from the row despite K-01. Stylistic, but the CLI table is the single source of exit-code truth.

Why it matters

Two implementers may or may not honor K-01's universal exit-2 for `run` usage errors.

Potential consequence

Inconsistent exit code, minor.

Recommended resolution

Add `/ 2 usage` to the `run` row (uniformity with the K-01 invariant).

F-015 — `by_difficulty` asymmetric presence unspecified

Severity: LOW

Location: R-05 (`by_category` MUST, `by_difficulty` optional) + C-06 (compare `n/m` semantics at metric level, not group level)

Observation

`by_difficulty` is optional on datasets; compare only defines metric-level `n/m` markers. What renders when one side carries `by_difficulty` and the other does not (group-level mismatch) is unspecified.

Why it matters

Presentation ambiguity only — both implementers produce *a* table, but shape differs.

Potential consequence

Cosmetic mismatch between output formats (annoying, not blocking).

Recommended resolution

Declare that compare treats group-level presence asymmetric as `n/m` (same marker extended), or document explicitly rendering “group absent”.

5. Requirements Review

Requirements (R-01..R-21) are in observability form: every statement names an actor, a condition, and a measurable artifact. The hierarchy (R-03, deterministic checks $\rightarrow$ judge $\rightarrow$ human validation) and the floor of stratification (R-05) are properly normative. Gaps:

- **R-02’s dataset-size floor (F-004)** cannot be enforced as specified; demote to a recommendation.
- **R-06’s Δ -table direction map** is one of the strongest requirement (concrete, direction-aware); F-015 surfaced only a minor cosmetical reservation.
- **R-11 (judge-check agreement) and R-13 (new-case sentinel)** are well-scoped to the §26/§28 semantics they encode. R-13 explicitly forbids fabricated ground truth, and yet the C-11 sentinel `REPLACE_ME` wording does not say which validator rejects it — F-012 group.

Missing requirements that would improve the spec: a normative requirement for **evaluator-ownership clarification** (F-001) and **status-enum mapping** (F-002); a compare-time **usage_kind guard** (F-013); and a **pair config schema** requirement (F-011).

6. Interface and Data-Contract Review

The contract level is high: C-01 dataset type with category closed set, C-02 pinned `ch3` adapter, C-03 verdict status totality, C-05 versioned `eval.json` literal (“0.1”), C-07 gates YAML, C-08 classifier precedence, C-09 label file format. Weak points:

- C-03’s status set vs `ch3`’s richer status (F-002); schema-level import of `ch3 verdict` would add ambiguity unless the mapping is declared explicitly.
- C-05 carries both `usage_kind` and a boolean that real costs come through only as `cost_usd`; compare-side guard missing (F-013).
- C-07 gates schema — validation of consistent keys is declared; pairing/units not pinned (F-005).
- C-09 label shape — label semantics (present-at-least-one-of-fields) adequate; schema reference absent in §11 (omitted from I-010’s list).
- **Dataset**’s lifecycle (one closure imposes `dataset_id` ambiguity — F-003).

Schema granularity is acceptable (jsonschema on every load); the schema-gate list (I-010) needs to include `pair config` and label files explicitly (F-011/E-10).

7. State and Failure Review

The §3.2 per-case state machine `LOADED  $\rightarrow$  INDEXED  $\rightarrow$  RUN  $\rightarrow$  CHECKED  $\rightarrow$  JUDGED  $\rightarrow$  METRIED  $\rightarrow$  CLASSIFIED` is correctly abstracted and total: `PARSE_BLOCKED` at `CHECKED`; judge skipped; strong detail. Failure semantics (§8) enumerate 18 rules covering usage errors, closure violations, zero-denominator, parse failures, run-time exceptions, schema drift, classification fallback, and availability taxonomies — a robust failure model.

Residual questions: whether `run` revalidates datasets (F-006), whether `PARSE_BLOCKED` zero-values propagate to metrics (F-007), and whether label evidence is ingested before or after classification (F-008). None is architecturally fatal; each is a semantic gap.

8. Determinism and Algorithm Review

Determinism is first-class: I-002 (byte-identity) with fixed `%.4f` rendering and sorted category keys; mock latencies are deterministic surrogates by contract (R-14); near-rank percentile method (K-05) pinned; the direction-map for Δ arithmetic centralized (I-004). The zero-denominator fallbacks (I-001/E-03) enumerate every metric in the `METRIC_KEYS` list with a named fallback — excellent.

The one residual algorithmic ambiguity is F-009: whether MRR/MAP/NDCG inherit `ch3`’s math or drop — directly affecting the deterministic direction map I-004. And F-007’s `PARSE_BLOCKED` numerics (completeness/accuracy values for blocked verdicts) is an algorithm-level gap.

9. Edge-Case Review

The §8 table is unusually broad (18 cases). It explicitly covers most of the systematic boundaries: empty dataset (E-03 fallbacks), uniform failure (E-05 to E-07), malformed input at two levels (E-02/E-14), pinned structural behaviors (E-12 dataset-id equality; E-08 stale-index). The F-012 findings show five edge cases that lack explicit tests (corpus-unread E-01, run-raise E-05, stale-index E-08, schema drift E-14, GUI malformed artifact E-17) — each is spec’d but untested, which is a traceability gap rather than a semantic one.

10. Non-Functional Requirement Review

Performance: K-02 names an explicit bound (<5 min per 100-case mock run) but “the host” is undefined hardware (commenting soft-target instead of hard-is correct — albeit weakly defined, LOW). Time-behavior is formally pinned (K-05 near-rank percentiles). Byte-determinism (I-002) and the `--mock` CI path are the right NFR for this type of subsystem.

Security (absent-as-appropriate): the harness’s own trust boundary is “documented” (§13) via the trust boundary between labels/judges and generation inputs (I-011 gold-isolation; I-016 adapter-only); external Ollama model-responsibility remains in `ch3` (E-13 carried). The spec is scoped to single-principal use; security NFR is adequate given stated scope (F-013 usage-kind risk is a side risk of data provenance rather than a security fault).

11. Security and Trust-Boundary Review

The harness’s threat surface is compact: (a) golden dataset files, (b) LLM/gate boundary (retrieved text as data, `ch3` R-21), (c) human label ingestion for evaluator validation, and (d) the `eval.json` provenance. Chapters `ch3` R-21 remains intact (retrieved evidence flagged via `injection_warning` through `citation.py`); gold-isolation I-011 forbids the evaluator’s expected-values from flowing into generation (a good trust boundary).

Remaining semantic points: labels are untrusted input for the classifier (F-008 ingestion timing); the `pair` config (JSON flags typed) lacks schema (F-011); the `eval.json` carries full `raw_output` text of the AoE — a comparatively small leak surface since the system reads local artifacts only (fine, and explicitly a non-issue in `ch3`). No privileged operations; no authentication. Your spec correctly scopes out an unrelated security architecture.

12. Observability and Provenance Review

Strongest section. The harness pins provenance: `dataset_id` name (set, F-003), trace record per `case_id` (C-02), `usage_kind` labeling (synthetic vs measured, R-14), capability deltas (`ch3` carried), and the R-to-I-to-C-to-E traceability matrix in §11 itself. Gaps are narrow:

- **F-012** (untested edge cases and collided T-ids) weakens the T-NN→module→behavior chain.
- **F-013** (usage-kind compare guard) hides dimension K provenance.

- **F-016** is technically avoided because protos are explicitly pinned to AoE results; good.

The §34 trace record is preserved per case — the class of mistake “what did class X do” is never guessy. The guide version 0.1 is ritualistically wired (E-06/E-14) and **check** tells the user which version they’re on.

13. Testing and Verification Review

Acceptance criteria are granular: T-01..T-24 grouped by §9.1 dataset, §9.1 pipeline, §9.3 metrics, §9.4 determinism, §9.5 compare/gates, §9.7 classification, §9.8 deliberate-regression (excellent exercise discipline), §9.9 judge validation, §9.10 optional surfaces, and §9.11 manual real-path smoke. For every requirement, the test that would prove it exists (**though F-012 undermines clarity by colliding ids**). Negatives (T-01b/T-01c/T-12/T-15b) and zero-denominator (T-05b) exercised.

What blocks “test-driven completeness”: T-01 assumes 5-row fixture (F-004), T-08a tests aim at the wrong verdict layer (F-001/F-002) — after which §9.3’s zero-denominator tests cease long-branch failover tests (F-007). The F-012 list names untested E-ids on which verification would be more secure.

14. Metrics and Evaluation Review

Formulas in C-04 are precise, named, and reference the right denominators per I-001 (groundedness = supported/F, completeness = reflected/total, hallucination = unsupported/total, latency near-rank percentile, cost per *successful* case). Reflected/relevant definition carries ch3’s F-006 interaction-with-reference only in comment form — below recommendation is to restate explicitly. Practical blocking bits: the **METRIC_KEYS normative closure** (F-009), **PARSE_BLOCKED numerics** (F-007), and missing **MRR/MAP/NDCG specification** (F-009) are the only metric-layer gaps.

Every metric is independently reproducible from evidence (trace + verdict) without requiring the component itself (the most important property; ch3 F-006 carried).

15. Traceability Review

The §11 matrix again is strong (each R mapped to a source/behavior + test); unfortunately it collapses on three ids: T-08a allocated to metrics and to judge-validation; T-13 to GUI and to pair; and a handful of E-ids (E-01, E-05, E-08, E-14, E-17) have no test row (F-012). That is a naming defect, not an architectural defect. §11 maintains its usefulness across P0 fixes if renumbered (recommended resolution §20).

16. Internal-Consistency Review

Internal consistency is currently the weakest axis:

- The ch4 status set restriction vs ch3’s PARTIAL-capable status (F-002) side-by-side is a true inconsistency.
- `AoEResult.verdict` ownership vs `evaluator.py` judges (F-001) is a true inconsistency.
- `dataset_id name/hash` destruction in `compare` (F-003).

Other sections are genuinely consistent: exit codes consistent globally (K-01) with a formatting outlier (F-014); category uniqueness (C-01) consistent with §21/§35 taxonomy; direction-map I-004 is one canonical source. Because consistency is required for Level 3, P0 items will close it out.

17. Architecture Review

The adapter architecture (`aoe.py` as the sole ch3 importer) is exactly the right shape; it makes the harness’s deterministic core formally decoupled from the ch3 rag import graph (I-016, source-scan-verified). The artifact-level architecture (six eval artifacts being the harness’s only durable outputs, §3.3) confines state to files, making `compare/gates`/GUI deterministic over artifacts, without inference (I-016/I-014). The component responsibilities are correct and the dependency direction (dataset → `aoe` → `evaluator` → metrics → `compare/gates` → GUI) never reverses. Architecture consists at Level 3 relatively established modulo P0.

18. Implementation-Agent Readiness

NO — MATERIAL QUESTIONS REMAIN.

Blocking questions (minimum set):

1. **Verdict ownership:** Does `AoEResult.verdict` become the evaluated verdict after deterministic checks, or is it re-judged by `evaluator.py`'s own `judges`? (F-001)
2. **Status mapping:** How does ch3's `PARTIAL` status map to ch4's `{PASS, FAIL, PARSE_BLOCKED}`? (F-002)
3. **Dataset idity:** Is `dataset_id` a declared name or a content hash? (F-003)
4. **Dataset size floor:** Does `check` enforce R-02's 50–100+ bound? (F-004)

Non-blocking clarifications (can be answered during implementation with proportionate cost): gates unit semantics (F-005), `run dataset` validation (F-006), `PARSE_BLOCKED` numerics (F-007), label-ingestion timing (F-008), report-only vs gates-addressable metric family (F-009), `--model` flag partitioning (F-010), pair config schema (F-011).

19. Quality Scorecard

Dimension	Score (0–5)	Evidence
Scope clarity	5	Single AoE boundary crisp (adapter), in/out scope explicit
Terminology	4	AoE/Evaluator/Judge/verdict/etc stable; <code>pair</code> label learns small drift
Requirement precision	4	Behavioral form mostly; R-02 floor lacks anchoring (F-004)
Interface completeness	3	Weak on the ch3-to-ch4 verdict seam (F-001/F-002)
Data-contract completeness	4	Lifecycle enumerated (C-01..C-11); <code>dataset_id</code> ambiguity
State/lifecycle definition	4	§3.2 state machine with explicit <code>PARSE_BLOCKED</code> path
Algorithm precision	4	Zero-denominator fallbacks + directionality (I-001/I-004)
Failure semantics	4	§8 table broad, I/O reset consistent (exits 0/1/2/3/4)
Edge-case coverage	4	E-01..E-18 listed, some untested (F-012)
Non-functional requirements	3	K-02 soft-typed as “host” partly unsupported; percentiles K-05 pinned
Security specification	3	Scope-appropriate; gold isolation and trust boundary normative

Dimension	Score (0–5)	Evidence
Observability/provenance	5	Trace per case, artifacts versioned, <code>usage_kind</code> labeled
Testability	4	Granular acceptance + regression fixtures (T-09) minus F-001/F-004 blockers
Evaluation/metrics	4	Formulas+denominators per I-001; F-009 metric-family ambiguity
Traceability	4	§11 matrix fine modulo F-012 ids
Internal consistency	3	F-001/F-002/F-003 real conflicts
Architecture consistency	4	Adapter+artifact pattern sound; I-016 enforced
Implementation readiness	3	Blocking questions listed above

Average: about 3.8 / 5

20. Remediation Plan

P0 — Blocking (must be resolved before implementation)

Finding	Resolution
F-001	Pin evaluator contract: AoE-provided verdict is the evaluated verdict; harness adds <code>DeterministicChecks</code> + classification only; judge wrappers restricted to the <code>judge-check</code> purpose
F-002	Declare the enum-mapping table: <code>PASS→PASS</code> , <code>FAIL→FAIL</code> , <code>PARTIAL→FAIL-with-nuance-flag</code> , <code>PARSE_BLOCKED</code> introduced by ch4 only; assert on load (I-010)
F-003	Resolve <code>dataset_id</code> semantics: declared name (not hash-of-content); compare guards on name equality; §28 case appends remain permitted
F-004	Demote R-02's 50–100+ to <code>SHOULD</code> (or gate behind a <code>--strict</code> flag); fix T-01 mode

P1 — Important

F-005 | Gates unit semantics: `max_pct_points` = absolute difference vs `max_pct` = relative-baseline change; pair-with-constraint validation; §25 MAY keys guarded |
F-006 | `run` dataset revalidation (recommended) with documented exit node (3) and warning option |
F-007 | `PARSE_BLOCKED` numeric defaults (`correct=false` etc., zero claims) |
F-008 | Label-ingestion timing — `judge-check` as post-run with optional backfill; classify which |
F-009 | `METRIC_KEYS` extension or report-only annotation for MRR/MAP/NDCG |
F-010 | `--judge-model/--gen-model` flag partition (or dedicated docs) |
F-011 | `schemas/pair.json` + I-010 gated-list update |

P2 — Improvement

F-012 | T-id renumbering (e.g. T-08a→T-08a/T-08c, T-13→T-13/T-24) + dedicated rows for E-01/E-05/E-08/E-14/E-17 |

F-013 | usage_kind compare warning/force |
F-014 | run exit row formatting uniform |
F-015 | by_difficulty group-asymmetry rendering rule |

Note: the §1 implies that all fifteen can be reviewed within one pass; P0 items are implementation-blocking, P1 items define conformance v0.2, P2 items are hygiene.

21. Final Verdict

Specification maturity:
Level 2 - Implementable

Implementation readiness:
NOT READY (material blocking questions remain)

Primary blocker:
Ambiguation of verdict ownership + verdict status mapping between the ch3 AoE and the ch4 evaluator layer (F-001/F-002).

Most important improvement:
Declare the ch3→ch4 verdict-status mapping explicitly and pin evaluator-ownership responsibility, then downgrade R-02's dataset-size floor to a recommendation (F-004).
